

Fe Trace / Content-Addressing Correctness Polish Review

Latest origin-overhaul-phased review and implementation directive - 2026-05-24

Scope and verification note

This is a static architecture and code-shape review of the latest Fe-side changes visible on origin-overhaul-phased. I did not run cargo, trybuild, the live LSP server, or the CLI locally in this sandbox. Treat build/test status as unverified until the project CI or an implementor workstation runs the command matrix in Appendix A.

The Sonatina-side trace/debug/content-addressing push is assumed to land before the correctness-polish work begins. This report therefore treats the current Fe-side Sonatina-surface bridge as transitional and gives concrete instructions for replacing it with the real Sonatina trace-view adapter once available.

1. Executive verdict

The latest Fe-side push is a meaningful step forward. The branch has moved from a fixture-only trace UX toward real compiler-derived trace emission: MIR now emits phase-owned local, storage, CFG, loop, and shape facts; codegen emits actual EVM bytecode instruction, opcode, category, gas, extent, code-object, and bytecode-shape facts; trace-query exposes gas, bytecode-size, PC explanation, variable-at-PC, optimized-code honesty, and loop/local reports; the LSP/live bridge and shape CLI are now present enough to start hardening them rather than merely imagining them.

The main architectural invariant still appears to be holding: phase-owned facts feed a canonical trace bundle, validators guard the bundle, query/report/LSP/CLI layers consume the bundle as derived views, and fixture-backed reports are still separated under fe dev trace-fixture.

My recommendation is not to add more feature surface before polish. The next Fe-side round should be a correctness and trust round: make reports harder to misread, strengthen data-source/confidence labeling, remove or quarantine transitional names that look more precise than they are, tighten query semantics, and prepare the Fe adapter to consume the incoming Sonatina trace substrate.

Area	Verdict	Action
Real trace emission	Strong progress. Real MIR and EVM bytecode facts now exist.	Keep; add confidence labels and source-span precision caveats.
Trace facts / validator	Much better than the old fact/relation sprawl. Validation is now broad.	Add remaining cross-fact consistency checks and duplicate/ambiguity handling.
Loop introspection	MIR/Sonatina-surface loop structures exist, but target loop contents are not yet proven.	Do not overclaim. Add loop-structure report now; final loop-cost waits for Sonatina-to-target membership.
Sonatina bridge	Current Fe-side bridge is useful but transitional.	Replace with adapter over the Sonatina PR trace views once landed.
Gas/bytecode by source	Useful coarse reports now possible.	Mark whole-file fallback as low-confidence until per-instruction source edges exist.
LSP/live bridge	Good scaffolding.	Harden token/security/status behavior before polish is called complete.
Content addressing	In the right direction.	Separate cryptographic content

		addresses from local FNV/stable fingerprints.
--	--	---

2. Current Fe-side state observed

The README now explicitly distinguishes fixture-backed Fibonacci demos from validated compiler-derived trace bundles. It says `fe dev trace-fixture` recognizes `fib_demo.fe` and emits fixture trace JSONL, while `fe dev trace` reads validated JSONL and reports the bundle data source from metadata. It also lists real trace gaps around loop membership, source-local labels, storage allocation, and zext causality. The source code has moved past part of that README wording: `trace_cli.rs` status now says real traces include source-local display names, MIR storage reasons, MIR lowering events, and value properties. Update the README to avoid stale caveats.

The current code organization under `crates/fe/src/trace` is a healthy split: `trace_cli.rs`, `trace_emit.rs`, `trace_fixture.rs`, `trace_live.rs`, and `trace_render.rs`. The CLI status path is now a useful truth label: fixture reports remain under `trace-fixture`, and compiler-derived reports read validated JSONL bundles.

`trace_emit.rs` validates emitted bundles before writing them. It builds a real trace by compiling the standalone input, emitting MIR facts, emitting codegen/Sonatina-surface facts, adding `SourceFile/SourceSpan` facts, compiling EVM bytecode, and emitting bytecode instruction/gas/shape facts. The tests check that real trace emission is compiler-emitted, validates, includes real instructions, includes shape hashes, and does not silently invent `LoopMembership` for real traces.

`mir::trace` now owns MIR/runtime facts directly. This is exactly the right ownership direction: MIR emits runtime functions, locals, display names, variables, storage decisions, lowering events, value properties, blocks, CFG edges, natural loop facts, instruction-block facts, and MIR loop shape facts. The code comments say MIR owns runtime statement/terminator identity while backend storage slots, registers, final instructions, and codegen events are emitted by codegen.

`codegen::trace` emits actual EVM bytecode facts from emitted bytes: code object, function, bytecode PC nodes, instruction facts, instruction extents, opcode/category facts, static gas facts, and shape facts. It also contains a Sonatina-surface bridge that mirrors MIR/runtime blocks into pre/post Sonatina-like keys, loop facts, CFG edges, instruction-block facts, snapshot identity edges, and shape hashes. That bridge is useful as a stopgap, but it must be treated as transitional until the Sonatina PR provides actual Sonatina trace views.

`trace-facts` has grown into the right canonical schema layer: origin nodes/edges, events, storage, instruction/category, block/CFG/loop facts, extents, inline context, opcode/gas, display names, value properties, source files/spans, code objects/functions/scopes/types/variables/location ranges, dynamic gas, and shape policy/hash facts. The new validating text wrappers for `OriginNodeKind`, `CompilerReason`, and `EvmSchedule` fix an earlier class of `serde-bypass` risk.

3. What is now strong enough to build on

- Fixture and real-trace paths are separate. This preserves trust and prevents the Fibonacci demo from pretending to be compiler evidence.
- MIR owns MIR facts. The CLI no longer needs to invent local-storage facts to explain `b`; it can read `MemoryPlace/SsaValue/Unknown` storage evidence from MIR.
- Bytecode/gas facts are emitted from actual EVM bytecode. Gas and bytecode-size reports are now grounded in real target artifacts, even if source attribution is still coarse.
- The validator is no longer a toy. It checks duplicate origin nodes, duplicate instruction sites, missing endpoints, block/function ownership, loop header roles, instruction extents, instruction categories, source spans, gas, dynamic gas consistency, and shape-policy/digest consistency.

- Shape facts are part of the trace snapshot rather than a separate reporting-only layer. That is the right path for content-addressed regression/fuzzing infrastructure.
- The CLI/LSP direction is coherent: fe dev trace works over bundles, fe dev trace live can query an LSP-hosted service, and the LSP server-info file is framed as a discovery pointer rather than a lock.

4. Correctness and polish directives

The following items are ordered by priority. P0 items should block declaring the Fe-side cleanup polished. P1 items should be done before presenting this as a reliable internal tool. P2 items are important follow-through once the Sonatina trace PR lands.

Priority	Fix	Why it matters	Concrete instruction
P0	Update README/status caveats	README currently says source-local labels and storage allocation are gaps, while current status/code now emit source-local display names and MIR storage facts. Stale docs undermine trust.	Rewrite status as: MIR labels/storage/value properties are emitted; backend storage, per-expression spans, zext causality, and target loop membership remain gaps.
P0	Make transitional Sonatina bridge explicit	emit_sonatina_* facts are currently derived from MIR/runtime structure, not actual sonatina_ir instrumentation.	Rename comments, status text, and report labels to "Sonatina-surface lowering-unit bridge" or "transitional Sonatina projection" until replaced by Sonatina trace views.
P0	Run and record full command matrix	This review is static. The branch must prove it builds and the demo commands work.	Run cargo check/test for fe, trace-facts, trace-query, mir, codegen, shape-address; run Appendix A CLI smoke matrix and attach output.
P0	Do not expose zext-report yet	This is already correctly gated. Keep it gated until backend/Sonatina emits InsertIntegerZeroExtend events and value-property facts that prove causality.	Keep zext-report unavailable in real trace; allow fixture-only wording if needed.
P1	Fix local-name ambiguity	TraceIndex maps display-name string to one OriginExportKey. Shadowing or duplicate names can silently pick the wrong local.	Index locals as name -> Vec<OriginExportKey>. explain-local should show ambiguity and accept --local-key / --owner / ordinal.
P1	Tighten PC lookup semantics	explain-pc currently falls back from bytecode pc:N to instruction.index == N, which can answer the wrong question for non-bytecode or multi-function traces.	Use bytecode PC lookup first. Require --index for instruction index fallback, or print "interpreted as instruction index" with low confidence.
P1	Label whole-file attribution as low confidence	Real trace currently has coarse whole-file SourceSpan fallback through code_object. Gas-by-source and bytecode-size-by-source can look more precise than they are.	Report attribution route: direct source, via code_object whole-file fallback, or unmapped. Whole-file fallback = Low confidence.
P1	Clarify gas attribution policies	call-inclusive and call-exclusive are parseable, but call graph semantics are not implemented yet.	Hide these policies or render "not yet call-aware; currently equivalent to inclusive/exclusive" until call facts exist.
P1	Cache origin reachability in	source_candidates_for_instr	Compute and cache

	TraceIndex	uction recomputes transitive reachability per instruction. This will hurt large traces and live LSP queries.	origin_reaches once per TraceIndex, or memoize per instruction.
P1	Separate fingerprint from content address	FNV-style hashes are acceptable for local fingerprints but not for trusted artifact identity or future package/integrity use.	Use BLAKE3/SHA-256 for exported content_hash/code_hash fields, or rename FNV fields to stable_fingerprint/local_fingerprint.
P1	Add JSON output for agent use	Sean and other agents should not scrape prose reports.	Add --format textljson for validate, loop-cost, explain-local, gas-by-source, bytecode-size-by-source, explain-pc, optimized-code-honesty.
P1	Harden LSP live endpoint and server-info lifecycle	The local HTTP/LSP bridge is powerful. It must not become a stale or spoofable trust surface.	Verify token required on every query; token file permissions 0600; stale .fe-lsp.json and .fe-lsp.token diagnostics; deterministic file selection.
P2	Add loop-structure report before loop-cost is fully available	MIR/Sonatina-surface loop facts already exist; target loop contents still wait on Sonatina-to-bytecode mapping.	Add fe dev trace query loop-structure that lists MIR/Sonatina loop blocks, roles, CFG hash, shape hashes, and says target membership unavailable.
P2	Replace Fe-side Sonatina surrogate with Sonatina adapter	Once Sonatina PR lands, Fe should consume actual trace views instead of maintaining mirrored Sonatina-like facts.	Implement codegen adapter: Sonatina function/block/inst/value -> TraceFacts; then deprecate transitional bridge.

5. Detailed fix areas

5.1 Documentation and honesty polish

The most important polish item is not code; it is trust. The branch is now more capable than the README says in some places and less capable than the new command surface might imply in others. Make the docs/status text match the exact evidence.

- Real trace now has MIR display names and MIR storage facts. Do not keep saying "source-local labels" or "storage allocation" are blanket gaps.
- Backend storage allocation remains a gap. "b is a MIR MemoryPlace" and "b is stack slot sp+24" are different claims. The former is available; the latter requires backend/Sonatina/target facts.
- Loop structure facts now exist at MIR and transitional Sonatina-surface layers. Target-level loop contents/cost remain unavailable until final bytecode PCs can be joined to loop membership.
- Gas and bytecode-size by source are currently coarse when they use whole-file code-object source span fallback. The reports should print that route and confidence.

Suggested status wording:

compiler-emitted: MIR locals/display names/storage/value properties, MIR CFG/loop/shape facts, transitional Sonatina-surface CFG/loop/shape facts, actual EVM bytecode/opcode/gas/extent/code-object facts.

coarse: source attribution currently falls back to whole-file code-object spans unless per-node source edges exist.

unavailable: target-level loop membership, Sonatina-to-bytecode emission edges, backend storage locations, and zext causality events.

5.2 Validator and schema correctness

The validator is now much stronger than earlier reviews anticipated. Keep it as the hard gate for every emitted trace. The remaining work is about semantic consistency rather than basic endpoint existence.

- Add a warning or error for duplicate DisplayNameFact entries for the same subject/kind with different names.
- Add a warning for duplicate human display names within the same function/scope so explain-local can surface ambiguity instead of silently picking one.
- Validate that LoopDerivation::NaturalLoopAnalysis cfg_hash is not empty. The shape-policy validator already rejects empty policy level/dimensions; give loop derivation the same treatment.
- Validate that ShapeGraphHash/ShapeNodeHash graph.owner kind is plausible for the graph level if ShapeGraphKey carries level information. This can be a warning initially.
- Consider validating code_hash/content_hash format once the digest algorithm is finalized. For now, at least document whether these are cryptographic digests or local fingerprints.
- Keep DynamicGasStep gas_cost == gas_before - gas_after validation for imported step facts, but document how refunds or aggregate transaction-level adjustments will be represented later. Step-level cost should stay simple.

5.3 Query/report correctness

Reports are where accidental overclaiming is most likely. The trace facts can be correct while the prose report is misleading. Every report row should carry enough confidence/provenance to tell the user whether it came from direct compiler facts, a posthoc classifier, a coarse fallback, or a fixture.

- explain-local should not be keyed only by display name. Display names are presentation, not identity. Return ambiguity when multiple locals share a name.
- explain-pc should avoid treating instruction index as PC without explicit user intent. This matters once non-bytecode, Sonatina, and Cranelift/native instructions share the trace snapshot.
- gas-by-source and bytecode-size-by-source should distinguish direct source attribution from code-object whole-file fallback.
- optimized-code-honesty should classify all compiler-overhead edge labels, not just SyntheticFor/BackendPrepared/Unmapped. Add IntegerLegalizationFor, SpillOf, ReloadOf, LoadOf, StoreOf where appropriate once those facts are emitted.
- variables-at-pc should state when no LocationRangeFact exists rather than implying no variables are available. The absence of location facts is a data gap, not proof of no variables.
- Reports should include: trace data source, classifier source, attribution policy, confidence, and missing facts needed to upgrade confidence.

Report	Current risk	Polish target
loop-cost	May be unavailable for real trace because no final LoopMembership exists.	Print unavailable reason plus available MIR/Sonatina-surface loop structures if present.
explain-local	Name ambiguity and backend-storage overclaim risk.	Display all candidate keys; separate MIR MemoryPlace from backend StackSlot.
gas-by-source	Whole-file source span can look like source-expression attribution.	Label as whole-file fallback / Low confidence.
bytecode-size-by-source	Same coarse attribution issue as gas.	Group direct, whole-file fallback, synthetic, unmapped buckets.
explain-pc	pc/index fallback can return wrong instruction.	Require explicit pc vs index mode.
optimized-code-honesty	Categories incomplete until Sonatina events exist.	Show missing event families and conservative unknown counts.

5.4 Content addressing and hash discipline

The content-addressing work is important enough to deserve stricter terminology. Shape hashes are content-addressed views. FNV-like `cfg_hash/code_hash/source` content hashes are local fingerprints unless they use a cryptographic digest and a versioned canonicalization policy.

- Use shape-address digests for IR/graph content-addressing where possible; they carry policy, dimensions, view mode, cycle policy, and graph key.
- Use BLAKE3 or SHA-256 for `SourceFileFact.content_hash` and `CodeObjectFact.code_hash` if those fields are intended to leave the test environment.
- If a fast non-cryptographic hash is useful for local CFG derivation, call it `cfg_fingerprint` or `local_fingerprint`, not `content_hash`.
- Every displayed hash should include or point to its policy: level, dimensions, algorithm, schema version, view mode, and cycle policy.
- Golden tests should assert dimension behavior: name-only change affects names dimension only; immediate-only change affects constants/immediates dimension; CFG edge change affects structure dimension; traversal-order noise does not change canonical aggregate.

5.5 Fe/Sonatina handoff plan

Because the Sonatina push has been handed off and is expected to land before these fixes are implemented, the Fe-side work should prepare to delete or demote the transitional Sonatina projection rather than expanding it.

1. Keep current MIR-derived Sonatina-surface facts only as a compatibility bridge while the Sonatina PR is in flight.
2. Once the Sonatina PR lands, add an adapter that consumes Sonatina trace views: function, block, instruction, value/debug handles, CFG edges, instruction-in-block, shape hashes, and optimization snapshot classifications.
3. Replace current pre/post identity-preserved facts with actual Sonatina pre/post snapshot facts. If the real Sonatina pass cannot prove a relationship, emit created/elided/unknown classifications rather than preserved identity.
4. Add target projection edges from Sonatina instructions/blocks to EVM bytecode PCs when available. Until then, bytecode facts remain target facts without source/MIR/Sonatina causality.
5. Only after Sonatina-to-target edges exist should real loop-cost claim target loop contents. Before that, provide loop-structure and target-wide gas/bytecode summaries.

Target chain after Sonatina PR lands:

```
MIR runtime block/stmt/local
-> Sonatina function/block/inst/value/debug handle
-> EVM bytecode PC/opcode/gas/byte-size
-> query reports: loop contents, gas by source, bytecode bloat, explain PC
```

5.6 LSP/live bridge correctness

The live LSP bridge is a high-value interface for agents, but it expands the trust boundary. It should be locked down before being sold as a polished capability.

- Require the localhost token on every trace-query endpoint. Reject missing/incorrect tokens even on loopback.
- Write token files with restrictive permissions on Unix. If Windows support matters, document equivalent behavior or avoid persisting reusable tokens.
- Make `.fe-lsp.json` a pure discovery pointer, never an authority. The server should validate PID, root URI, config hash, token path, and freshness before trusting it.
- If live query has to compile a file automatically, deterministic file selection matters. Sort recursive candidates, or require an explicit URI when multiple Fe files exist.

- LSP status/doctor should diagnose stale server-info, stale token file, root mismatch, malformed JSON, missing trace API URL, and unsupported capabilities.

6. Phased implementation plan for correctness polish

Phase	Goal	Concrete deliverables	Exit criteria
0. Build gate	Prove the branch is mechanically healthy.	Cargo check/test matrix; CLI smoke scripts; fixture and real trace outputs captured.	All commands pass or known failures are documented with issue IDs.
1. Honesty pass	Make all docs/reports match actual evidence.	README/status updates; report confidence headers; whole-file fallback labels; Sonatina bridge caveat labels.	No report claims backend storage, zext causality, or target loop membership without corresponding facts.
2. Query correctness	Eliminate ambiguous query behavior.	Name ambiguity handling; --local-key; explicit --pc vs --index; source attribution route/confidence; call policy gating.	Ambiguous locals and coarse source attribution are visible in text/JSON.
3. Validator polish	Catch remaining drift before reports run.	Display-name duplication warnings; loop derivation hash checks; digest format policy; shape level consistency warnings.	trace validate is the mandatory precondition for every report.
4. Agent/LSP polish	Make the tools useful and safe for agents.	--format json; live endpoint token enforcement; status/doctor refinements; deterministic discovery.	Sean/agent can consume structured results without scraping prose.
5. Sonatina adapter	Replace transitional Sonatina projection with real Sonatina trace views.	Adapter over Sonatina PR; real Sonatina function/block/inst/value facts; real snapshot classifications; target projection edge hook.	Current transitional bridge is removed or explicitly legacy-only.

7. Acceptance checklist

Call the Fe-side correctness round complete only when these are true:

- README, fe dev trace status, and report headers agree on what is compiler-emitted, fixture-backed, posthoc, coarse, unavailable, or transitional.
- Every fe dev trace report validates the input bundle before reading it.
- No report uses a display name as identity without disambiguation.
- explain-pc cannot silently confuse bytecode PC with instruction index.
- gas-by-source and bytecode-size-by-source label whole-file code-object fallback as low confidence.
- content_hash/code_hash fields are cryptographic or renamed as local fingerprints.
- trace-query caches reachability or otherwise avoids recomputing transitive closure per instruction in normal reports.
- LSP/live trace queries require a token and diagnose stale/mismatched server-info files.
- The Fe-side Sonatina surrogate is either clearly labeled transitional or replaced by the real Sonatina trace adapter.
- A JSON output mode exists for the core agent-facing reports.
- Golden tests cover real trace emit/validate/gas-by-source/bytecode-size/explain-local/explain-pc, plus fixture trace sanity.

Appendix A. Command matrix to run before polish sign-off

Run these on an implementor workstation or CI. Capture stdout/stderr and attach the results to the review notes. Adjust package/feature names if the workspace layout changes.

```
# Mechanical checks
cargo check -p trace-facts
cargo check -p trace-query
cargo check -p shape-address
cargo check -p mir
cargo check -p codegen
cargo check -p fe
cargo test -p trace-facts
cargo test -p trace-query
cargo test -p shape-address
cargo test -p mir
cargo test -p codegen
cargo test -p fe

# Real compiler-derived trace
cargo run -p fe -- dev trace emit fib_demo.fe --out target/fib.trace.jsonl
cargo run -p fe -- dev trace validate --from target/fib.trace.jsonl
cargo run -p fe -- dev trace loop-cost --from target/fib.trace.jsonl
cargo run -p fe -- dev trace explain-local --from target/fib.trace.jsonl --local b
cargo run -p fe -- dev trace query gas-breakdown --from target/fib.trace.jsonl
cargo run -p fe -- dev trace query gas-by-source --from target/fib.trace.jsonl
cargo run -p fe -- dev trace query bytecode-size-by-source --from target/fib.trace.jsonl
cargo run -p fe -- dev trace query explain-pc --from target/fib.trace.jsonl --pc 0
cargo run -p fe -- dev trace query optimized-code-honesty --from target/fib.trace.jsonl

# Fixture remains explicitly fixture-backed
cargo run -p fe -- dev trace-fixture emit fib_demo.fe --out target/fib.fixture.trace.jsonl
cargo run -p fe -- dev trace validate --from target/fib.fixture.trace.jsonl
cargo run -p fe -- dev trace-fixture loop-cost fib_demo.fe
cargo run -p fe -- dev trace-fixture explain-local fib_demo.fe --local b

# Shape/content-addressing smoke checks
cargo run -p fe -- shape emit --bytecode-hex 0x5f600101
cargo run -p fe -- shape explain --bytecode-hex 0x5f600101
cargo run -p fe -- shape diff --left-bytecode-hex 0x5f600101 --right-bytecode-hex 0x5f600201
cargo run -p fe -- shape bucket --bytecode-hex 0x5f600101

# LSP/live smoke, if feature flags/environment support it
cargo run -p fe -- lsp status
cargo run -p fe -- lsp doctor
cargo run -p fe -- lsp config --show
cargo run -p fe -- dev trace live status
cargo run -p fe -- dev trace live query status
```

Appendix B. Suggested report header format

```
Trace bundle: target/fib.trace.jsonl
Data source: compiler_emitted | fixture
Compiler profile: dev / 01
Target: evm/sonatina
Validation: passed, N facts, M nodes, K instructions
Attribution route: direct_source | code_object_whole_file_fallback | unmapped
Classification source: backend_emission_reason | posthoc_classifier | manual_fixture
Confidence: high | medium | low | unknown
Missing facts to improve confidence: [sonatina_to_bytecode_edges, backend_storage_locations,
zext_events]
```

Reports should use this metadata before findings. A human or agent should never need to infer whether an answer is real, fixture-backed, posthoc, coarse, or unavailable.

Appendix C. Sources reviewed

Path	Reviewed for
README.md	Developer Trace Prototype section: fixture vs real trace commands and stated gaps.
crates/fe/src/main.rs	Shape CLI, DevTrace commands, trace query commands, LSP server-info behavior.
crates/fe/src/trace/trace_cli.rs	Trace-fixture vs trace dispatch; status text; query dispatch.
crates/fe/src/trace/trace_emit.rs	Real trace emit/validate path; query wrappers; real-trace tests.
crates/mir/src/trace.rs	MIR-owned locals/storage/CFG/loop/shape trace emission.
crates/codegen/src/trace.rs	EVM bytecode facts; gas facts; bytecode shape facts; transitional Sonatina-surface bridge.
crates/trace-facts/src/fact.rs	Expanded trace schema and validating text wrappers.
crates/trace-facts/src/validate.rs	Trace validator checks and diagnostics.
crates/trace-query/src/lib.rs	Introspection service, query reports, TraceIndex implementation.

Public branch URL reviewed: <https://github.com/micahscopes/fe/tree/origin-overhaul-phased>

Raw/source inspection note: GitHub HTML sometimes compresses raw files into long single-line views. The validator and query files were also inspected from raw source copies where possible.

Appendix D. Handoff expectations once Sonatina PR lands

The Fe-side correctness work should assume the Sonatina PR supplies frontend-neutral debug/provenance handles, trace-visible function/block/instruction/value views, CFG edges, instruction-in-block, shape hashes, and conservative debug/optimization propagation. Fe should consume those views; Sonatina should not depend on Fe `OriginExportKey` directly.

Fe-side need	Before Sonatina PR	After Sonatina PR
Sonatina function/block/inst facts	Transitional MIR-derived projection.	Adapter from actual Sonatina trace views.
Loop structure	MIR and Sonatina-surface loops available.	Actual Sonatina CFG loops, shape hashes, instruction membership.
Target loop contents	Unavailable without target membership.	Derive once Sonatina-to-EVM bytecode PC edges exist.
Optimizer honesty	Conservative snapshot placeholder.	Actual pre/post snapshot classifications.

Content addressing	MIR/bytecode/transition shape hashes.	Actual Sonatina function/block/loop shape hashes.
Other frontends	Fe-specific adapter only.	Frontend-neutral handles allow other frontends to attach provenance.